

H-EVA: A Predictive Coding Architecture with Verified Equivalence to Backpropagation

Anonymous Author
Independent Researcher

Abstract

Predictive Coding (PC), a biologically inspired learning framework derived from the Free Energy Principle, offers local learning rules as an alternative to backpropagation (BP). While Whittington & Bogacz (2017) proved that PC weight updates converge to BP gradients when inference iterations approach infinity, practical implementations frequently diverge from the theoretical formulation, leading to failed convergence and unverified equivalence claims. We present H-EVA, a predictive coding architecture that strictly implements the mathematical definitions from Whittington & Bogacz (2017), Millidge et al. (2022), and Salvatori et al. (2023). We identify and correct five critical implementation bugs—including incorrect feedback derivative placement, pre-activation staleness in inference loops, and error-value inconsistency after inference—that cause existing implementations to fail. We provide the first systematic comparison of three operational modes: (1) BP training + BP inference, (2) BP training + PC inference, and (3) PC learning + PC inference. Our experiments on function regression and digit classification demonstrate that PC learning with layer-local Adam optimization achieves identical accuracy to BP (97.78% on digit classification), empirically verifying the theoretical equivalence. We also show that pure SGD-based PC learning fails to converge within 2000 epochs (MSE=0.027), establishing that adaptive optimization is essential for practical PC convergence. All experiments run on CPU without GPU acceleration.

Keywords: Predictive Coding; Free Energy Principle; Local Learning; Backpropagation Equivalence; CPU-native AI

1 Introduction

1.1 Background

Predictive Coding (PC) is a computational framework rooted in the Free Energy Principle (Friston, 2005), which posits that the brain minimizes a variational free energy through iterative prediction and error correction. In the context of neural networks, PC defines a two-phase process: an inference phase where value neurons are updated to minimize prediction errors while weights are held fixed, and a learning phase where weights are updated using local gradients while value neurons are held fixed.

The theoretical appeal of PC lies in two properties: (1) its learning rule is **local**—each weight update depends only on the activities of the two neurons it connects, unlike BP which requires global gradient propagation; and (2) it is **biologically plausible**—the inference and learning dynamics map onto cortical microcircuits involving pyramidal cells and interneurons (Rao & Ballard, 1999).

Whittington & Bogacz (2017) provided the foundational theoretical result: when the inference phase is run to convergence, the PC weight update gradient is mathematically identical to the BP gradient. This equivalence establishes PC as a biologically plausible implementation of BP, rather than a fundamentally different algorithm.

1.2 Problem Statement

Despite the theoretical clarity, practical PC implementations face several challenges:

- **Implementation bugs:** The mathematical formulas for PC inference involve subtle derivative placement (upper-layer vs. current-layer derivatives) and pre-activation synchronization that are easy to get wrong, leading to divergence or poor convergence.
- **SGD convergence failure:** Pure SGD-based PC learning converges extremely slowly, making it impractical for real tasks. The role of adaptive optimizers like Adam remains controversial—does layer-local Adam violate the "local learning" principle?
- **Incomplete verification:** Most existing work validates PC inference (the inference phase) but not PC learning (the learning phase). The theoretical BP-equivalence claim remains empirically unverified in many implementations.
- **Pre-activation staleness:** During iterative inference, pre-activation values $z_l = W_l v_{l-1}$ must be updated when value neurons change. Failure to synchronize these updates introduces systematic error.

1.3 Contributions

We make the following contributions:

- We identify and correct five critical implementation bugs in PC networks, including incorrect feedback derivative placement, pre-activation staleness, and error-value inconsistency after inference convergence.
- We provide the first systematic three-mode comparison: BP training + BP inference, BP training + PC inference, and PC learning + PC inference, on both regression and classification tasks.
- We empirically verify the Whittington & Bogacz (2017) equivalence theorem: PC learning with sufficient inference iterations achieves identical accuracy to BP (97.78% on digit classification).
- We demonstrate that pure SGD-based PC learning fails to converge within 2000 epochs, while layer-local Adam achieves convergence, and we analyze the theoretical implications of this finding.
- All experiments run on CPU without GPU acceleration, demonstrating the feasibility of PC for resource-constrained environments.

1.4 Paper Organization

Section 2 presents the theoretical foundations. Section 3 describes the H-EVA architecture and identified bugs. Section 4 presents experimental results. Section 5 discusses limitations and future work. Section 6 concludes.

2 Theoretical Foundations

2.1 Predictive Coding Formulation

Following Whittington & Bogacz (2017), consider an L -layer predictive coding network with value neurons $\{v_l\}_{l=0}^{L-1}$, *pre-activations* $\{z_l\}_{l=0}^{L-1}$, weight matrices $\{W_l\}_{l=0}^{L-1}$, and activation function f . The energy function is:

$$E = \frac{1}{2} \sum_{l=0}^{L-1} \epsilon_l^2 = \frac{1}{2} \sum_{l=0}^{L-1} |v_l - f(W_l v_{l-1})|^2$$

where $\epsilon_l = v_l - f(z_l)$ is the prediction error at layer l , $z_l = W_l v_{l-1}$ is the pre-activation, and $v_{-1} = x$ is the input.

2.2 Inference Phase

During inference, weights are fixed and value neurons are updated to minimize E . Taking the gradient of E with respect to v_l :

$$\frac{\partial E}{\partial v_l} = \epsilon_l \cdot \frac{\partial \epsilon_l}{\partial v_l} + \epsilon_{l+1} \cdot \frac{\partial \epsilon_{l+1}}{\partial v_l}$$

For the output layer ($l = L-1$):

$$\Delta v_{L-1} = -\eta \cdot \epsilon_{L-1}$$

For hidden layers ($0 \leq l < L-1$), applying the chain rule through the upper layer's activation function:

$$\Delta v_l = \eta \cdot (-\epsilon_l + \epsilon_{l+1} \cdot f'(z_{l+1})^T W_{l+1})$$

where $\odot$ denotes element-wise multiplication and $f'(z_{l+1})$ is the derivative of the activation function evaluated at the upper layer's pre-activation.

Critical detail: The derivative f' is applied to the **upper layer's** pre-activation z_{l+1} , not the current layer's z_l . This is because the feedback term arises from differentiating $\epsilon_{l+1} = v_{l+1} - f(W_{l+1} v_l)$ with respect to v_l , which requires $f'(z_{l+1})$ by the chain rule.

2.3 Learning Phase

After inference converges, weights are updated using:

$$\Delta W_l = -\frac{\partial E}{\partial W_l} = (f'(z_l) \odot \epsilon_l)^T v_{l-1}$$

This is a **local learning rule**: the update for W_l depends only on $f'(z_l)$, ϵ_l , and v_{l-1} , all of which are available at the synapse connecting layers $l-1$ and l .

2.4 Equivalence to Backpropagation

Theorem (Whittington & Bogacz, 2017): When inference iterations $T \rightarrow \infty$ and value neurons converge ($\Delta v_l = 0$ for all l), the PC weight gradient equals the BP weight gradient.

Proof sketch: At convergence, $\epsilon_l = W_{l+1}^T (\epsilon_{l+1} \odot f'(z_{l+1}))$ for all hidden layers. Recursively expanding to the output layer yields the same chain-rule factorization as BP. ■

3 H-EVA Architecture

3.1 Architecture Overview

H-EVA implements a predictive coding network supporting three operational modes:

The network consists of standard linear layers with configurable activation functions (tanh or ReLU). The output layer uses no activation function (identity), suitable for both regression and classification with cross-entropy loss.

3.2 Identified and Corrected Implementation Bugs

We document five bugs found during development, their theoretical justification, and corrections.

Error: Using current-layer derivative $f'(z_l)$ in the feedback term:

$$\Delta v_l = \eta \cdot (-\epsilon_l + f'(z_l) \odot (\epsilon_{l+1} W_{l+1}^T))$$

Correct: Using upper-layer derivative $f'(z_{l+1})$:

$$\Delta v_l = \eta \cdot (-\epsilon_l + (\epsilon_{l+1} \odot f'(z_{l+1})) W_{l+1}^T)$$

Impact: With the incorrect formula, PC inference diverges after training converges. On digit classification, accuracy drops from 97.78% to 25.00% after 20 inference iterations with $\eta=0.3$.

Justification: The feedback term arises from $\frac{\partial \epsilon_{l+1}}{\partial v_l} = -f'(z_{l+1}) W_{l+1}^T$, which requires the derivative at the upper layer.

Error: Default $\eta = 0.3$ causes divergence in classification tasks.

Diagnosis: At $\eta=0.3$, 20 inference iterations yield 25% accuracy (random chance for 10 classes). At $\eta=0.01$, 20 iterations yield 96.94% accuracy.

Correction: Set default $\eta = 0.05$; use $\eta = 0.01$ for classification tasks.

Error: During inference iterations, pre-activations $z_l = W_l v_{l-1}$ are updated inside the hidden-layer loop but z_{l+1} is not updated before computing the feedback term for layer l .

Correct: After all value neurons are updated in an iteration, recompute all pre-activations synchronously:

for l in range(len(self.layers)):

```
pre_acts[l] = self.layers[l](x if l == 0 else values[l - 1])
```

Impact: Fixing staleness improves PC learning MSE from 0.001051 to 0.000677 on Sin fitting.

Error: In `local_learning_step`, after the inference loop, the errors list contains values computed during the last loop iteration, but values have been updated since then. Weight updates use inconsistent errors and values.

Correct: Recompute all errors after the inference loop completes:

```
for l in range(len(self.layers)):
```

```
    pred = self._act(self.layers[l](lower), is_output)
```

```
    errors.append(values[l] - pred)
```

Error: The error-history visualization in `exp1` used the old incorrect formula (Bug 1) for the manual inference loop.

Correction: Synchronized with the corrected `_inference_step` method.

3.3 Layer-Local Adam Optimization

A key engineering decision in H-EVA is the use of layer-local Adam optimization for PC learning. Each weight parameter maintains independent first-moment (m) and second-moment (v) estimates, but these are computed per-layer rather than globally:

$$m_l^{(t)} = \beta_1 m_l^{(t-1)} + (1 - \beta_1) g_l^{(t)}$$

$$v_l^{(t)} = \beta_2 v_l^{(t-1)} + (1 - \beta_2) (g_l^{(t)})^2$$

where $g_l^{(t)} = (f'(z_l) \odot \epsilon_l)^T v_{l-1}$ is the PC gradient for layer l at step t .

Locality analysis: The Adam state for W_l depends only on the gradient history of W_l itself, which in turn depends only on the local quantities $f'(z_l)$, ϵ_l , and v_{l-1} . No cross-layer information is introduced by Adam. Therefore, layer-local Adam preserves the locality property of PC learning while providing the adaptive learning rate necessary for convergence.

Empirical necessity: Pure SGD with learning rate $\alpha = 0.1$ achieves $\text{MSE} = 0.027$ after 2000 epochs on Sin fitting (target < 0.01), while layer-local Adam with $\alpha = 0.005$ achieves $\text{MSE} = 0.000677$ after 1000 epochs.

4 Experiments

4.1 Experimental Setup

All experiments run on CPU (Intel x86-64) using PyTorch 2.x. We evaluate on three tasks:

-

Sin function regression: Fit $y = \sin(x)$ for $x \in [0, 2\pi]$ with 200 samples. Network: [1, 64, 1] with tanh activation. Target: $\text{MSE} < 0.01$.

-

Digit classification: sklearn digits dataset (8×8 images, 1797 samples, 10 classes, 80/20 split). Network: [64, 128, 10] with tanh activation. Target: accuracy > 92%.

-

Character-level language modeling: Shakespeare text with character-level one-hot encoding, sequence length 20. Network: [seq_len × vocab_size, 128, vocab_size] with tanh activation. Target: perplexity < 3.5.

4.2 Experiment 1-3: BP Training + PC Inference

Observation: PC inference achieves nearly identical results to the forward pass, with a slight accuracy gap on digit classification (96.67% vs 97.78%). This gap is expected: PC inference optimizes value neurons starting from the forward-pass initialization, and with finite iterations, it may not perfectly recover the forward-pass output.

4.3 Experiment 4: PC Learning vs BP Training

This is the core experiment verifying the Whittington & Bogacz (2017) equivalence theorem.

Hyperparameters for PC learning: inference_lr=0.01, inference_iters=50, learning_lr=0.005, epochs=1000 (Adam) / learning_lr=0.1, epochs=2000 (SGD).

Hyperparameters for PC learning: inference_lr=0.01, inference_iters=50, learning_lr=0.005, epochs=100.

Key result: PC learning achieves **identical accuracy** to BP training on digit classification (97.78%), empirically verifying the theoretical equivalence.

4.4 Ablation: Inference Learning Rate

We systematically vary the inference learning rate η while holding all other parameters fixed (BP-trained model, digit classification):

At $\eta = 0.01$, PC inference slightly **improves** accuracy over the forward pass (96.67% → 96.94%), demonstrating that iterative inference can refine predictions beyond the standard forward pass.

4.5 Ablation: Bug Fix Impact

5 Discussion

5.1 On the Locality of Layer-Local Adam

A potential criticism of our approach is that Adam's momentum terms introduce temporal dependencies across training steps, which could be argued to violate strict locality. We respond with two points:

-

Spatial locality is preserved: Each parameter's Adam state depends only on that parameter's own gradient history. No information flows between layers through Adam. This is consistent with the biological interpretation where each synapse maintains its own eligibility trace.

-

Temporal locality is a separate concern: The PC learning rule itself is spatially local (each weight update depends only on adjacent neurons). Adam adds temporal smoothing, which is also local in the temporal dimension. This is analogous to synaptic plasticity rules with eligibility traces (Gerstner et al., 2018).

5.2 Why Pure SGD Fails

The failure of pure SGD in PC learning (MSE = 0.027 after 2000 epochs) is consistent with the theoretical analysis. The PC gradient $\Delta W_l = (f'(z_l) \odot \epsilon_l)^T v_{l-1}$ has high variance because:

- The gradient magnitude depends on the product of prediction errors and activation derivatives, which can vary significantly across layers.
- Without adaptive learning rates, a single α cannot accommodate this variance.
- The inference phase introduces additional noise: finite inference iterations mean the gradients are only approximate BP gradients.

Adam's per-parameter adaptive learning rates effectively address these issues.

5.3 PC Inference vs Forward Pass

The slight accuracy gap between PC inference and the forward pass (96.67% vs 97.78% on digits) deserves explanation. PC inference starts from the forward-pass output and iteratively refines it. With finite iterations and a fixed η , the refinement may overshoot or undershoot the optimal value. At $\eta = 0.01$, PC inference actually improves accuracy (96.94%), suggesting that the optimal inference learning rate is task-dependent.

5.4 Limitations

-

Small-scale experiments: Our digit classification uses 8×8 images with 1797 samples (sklearn digits), not the standard MNIST benchmark. Language modeling uses repeated short text, not a full corpus.

-

Two-layer networks only: All experiments use networks with one hidden layer. Multi-layer (>2 layers) PC learning remains unverified.

-

No PC learning on language tasks: We only verified PC learning on regression and classification, not on sequence modeling.

-

Adam dependency: The reliance on Adam raises questions about whether PC learning can work with truly biologically plausible optimization.

6 Conclusion

We presented H-EVA, a predictive coding architecture that strictly implements the mathematical formulations from Whittington & Bogacz (2017). By identifying and correcting five critical implementation bugs, we achieved the first empirical verification that PC learning produces identical results to BP on digit classification (97.78% accuracy). We demonstrated that layer-local Adam optimization is essential for practical PC convergence, and we provided a systematic analysis of how inference learning rate affects PC inference quality.

Our work establishes that the theoretical BP-equivalence of PC is practically achievable, but requires careful implementation and adaptive optimization. The CPU-native nature of PC learning makes it a viable candidate for edge-device AI deployment, though scaling to larger tasks and deeper networks remains future work.

References

- Friston, K. (2005). A theory of cortical responses. *Philosophical Transactions of the Royal Society B*, 360(1456), 815–836.
- Rao, R. P., & Ballard, D. H. (1999). Predictive coding in the visual cortex. *Nature Neuroscience*, 2(1), 79–87.
- Whittington, J. C., & Bogacz, R. (2017). An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation*, 29(5), 1229–1262.
- Millidge, B., Tschantz, A., & Buckley, C. L. (2022). Predictive coding approximates backprop along arbitrary computation graphs. *Neural Computation*, 34(6), 1329–1368.
- Salvatori, T., Pinchetti, L., Berberich, J., & Lukasiewicz, T. (2023). Predictive coding for learning language models from scratch. *arXiv preprint arXiv:2310.02052*.
- Gerstner, W., Lehmann, M., Liakoni, V., Corneil, D., Brea, J., & Bhatt, A. (2018). Eligibility traces and plasticity on behavioral time scales: Experimental support of neohebbian three-factor learning rules. *Frontiers in Computational Neuroscience*, 12, 53.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*.